Tarea 1 · Biblioteca del IES (UT1, UT2, UT4)

Oct 21, 2026 · @Carlos

Presentación

Esta tarea repasa todo lo visto hasta ahora: integración de JavaScript en una página, sintaxis básica, arrays, Set, Map, funciones y métodos de arrays con callback. Es individual y se divide en dos partes.

- **Parte A — Calentamiento.** Seis ejercicios cortos para comprobar que dominas cada concepto por separado.
- **Parte B — Reto integrador.** Una pequeña aplicación de consulta del fondo de la biblioteca del instituto, con HTML y CSS básicos.

Duración orientativa: 6 horas lectivas (2 para la parte A y 4 para la parte B), más trabajo autónomo.

Qué tienes que conseguir:

1. Predecir y comprobar en la consola cómo se comporta JavaScript con tipos, conversiones y ámbitos.
2. Resolver problemas con decisiones y bucles.
3. Usar arrays, Set y Map para organizar datos.
4. Escribir funciones propias pequeñas, reutilizables y documentadas.
5. Procesar colecciones con `filter`, `map`, `reduce`, `sort` y `forEach`.
6. Depurar tu código con las herramientas de desarrollo del navegador.

Relación curricular

La tarea aporta evidencias de 14 criterios de evaluación de RA01, RA02 y RA04. La redacción de los criterios es literal de la programación didáctica.

UT	RA	Criterio de evaluación
UT1	RA01	e) Se han verificado los mecanismos de integración de los lenguajes de marcas con los lenguajes de programación de clientes Web
UT1	RA01	f) Se han reconocido y evaluado las herramientas de programación sobre clientes Web
UT2	RA02	b) Se han utilizado los distintos tipos de variables y operadores disponibles en el lenguaje
UT2	RA02	c) Se han identificado los ámbitos de utilización de las variables
UT2	RA02	d) Se han reconocido y comprobado las peculiaridades del lenguaje respecto a las conversiones entre distintos tipos de datos
UT2	RA02	e) Se han utilizado mecanismos de decisión en la creación de bloques de sentencias
UT2	RA02	f) Se han utilizado bucles y se ha verificado su funcionamiento

UT	RA	Criterio de evaluación
UT2	RA02	g) Se han añadido comentarios al código
UT2	RA02	h) Se han utilizado herramientas y entornos para facilitar la programación, prueba y depuración del código
UT4	RA04	a) Se han clasificado y utilizado las funciones predefinidas del lenguaje
UT4	RA04	b) Se han creado y utilizado funciones definidas por el usuario
UT4	RA04	c) Se han reconocido las características del lenguaje relativas a la creación y uso de arrays
UT4	RA04	d) Se han creado y utilizado arrays
UT4	RA04	i) Se ha depurado y documentado el código

Parte A — Calentamiento

Resuelve cada ejercicio en un archivo `ejA1.js`, `ejA2.js`… cargado desde un `index.html` propio, y comprueba los resultados en la consola. Las respuestas escritas van en `respuestas.md`.

A1. Conversiones de tipos

Antes de ejecutar nada, escribe tu **predicción** para cada expresión. Después compruébala en la consola y completa la tabla en `respuestas.md`.

```
"5" + 3
"5" * 3
"5" - "2"
Number("")
Number("12px")
parseInt("12px")
parseFloat("3.75€")
"10" == 10
"10" === 10
null + 1
undefined + 1
typeof NaN
Boolean("0")
0 || "valor por defecto"
0 ?? "valor por defecto"
```

Expresión	Mi predicción	Resultado real	Tipo (`typeof`)	Explicación (una línea)
`"5" + 3`				

Para terminar, responde: ¿por qué `NaN === NaN` devuelve `false`? ¿Cómo compruebas correctamente si un valor es `NaN`?

A2. Ámbito de las variables

Analiza este código **sin ejecutarlo**:

```
const gastosEnvio = 5;
let total = 0;

function calcularTotal(precio, unidades) {
  let total = precio * unidades + gastosEnvio;
  if (total > 50) {
    const descuento = 10;
    total = total - descuento;
  }
  console.log("Dentro:", total);
  return total;
}

calcularTotal(20, 3);
console.log("Fuera:", total);
console.log(descuento);
```

1. ¿Qué muestra cada `console.log`? Ejecútalo después y compara.
2. ¿Por qué `total` vale distinto dentro y fuera de la función?
3. ¿Qué error produce la última línea y por qué?
4. ¿Qué cambia si dentro de la función sustituyes `let total = …` por `total = …`?
5. ¿Qué ocurre si añades `gastosEnvio = 7;` al final del programa?

A3. Decisiones y bucles: boletín de notas

Con este array:

```
const notas = [7.5, 4.9, 10, 5, 6.25, 3, 8.8, 11, 9];
```

1. Recorre el array con un bucle `for` clásico y muestra la posición y la calificación de cada nota: `Alumno 1: 7.5 → Notable`.
   - Insuficiente (< 5), Suficiente (< 6), Bien (< 7), Notable (< 9), Sobresaliente (≤ 10).
   - Una nota fuera del rango 0–10 se muestra como `Nota no válida` y no cuenta en los cálculos.
2. Con un bucle `for...of`, calcula el número de aprobados, la media de las notas válidas y la nota más alta. No uses `Math.max` ni métodos de arrays en este ejercicio.

A4. Set y Map: lista de inscripciones

```
const inscripciones = [
  "ana@ies.es", "luis@ies.es", "ANA@ies.es", "marta@ies.es",
  "luis@ies.es", "pablo@ies.es", "Marta@ies.es"
];
```

1. Usa un `Set` para obtener los correos sin repetir. ¿Cuántos salen? ¿Por qué no es el resultado que esperabas?
2. Repite el apartado normalizando antes cada correo a minúsculas (`correo.toLowerCase()`). ¿Cuántos salen ahora?
3. Usa un `Map` para contar cuántas veces aparece cada correo normalizado. Recórrelo y muestra `luis@ies.es → 2`.
4. Convierte el `Set` del apartado 2 en un array ordenado alfabéticamente.

A5. Funciones y callbacks

1. Escribe `transformar(array, fn)`, que devuelve un array nuevo aplicando `fn` a cada elemento. Usa un bucle, no `map`.
2. Escribe `quedarse(array, condicion)`, que devuelve un array nuevo con los elementos que cumplen la condición. Usa un bucle, no `filter`.
3. Pruébalas con funciones flecha: duplica los números de `[3, 8, 1, 6]` y quédate con los pares.
4. Reescribe el apartado 3 con `map` y `filter`. Explica en dos líneas qué tienen en común tus funciones y los métodos de los arrays.
5. Con `const precios = [12, 45, 8, 99, 30];`, encadena métodos para obtener el total con IVA (21 %) de los productos de más de 10 €, redondeado a dos decimales.

A6. Depuración

Esta función debería devolver la **media de las notas aprobadas** (5 o más), pero contiene varios errores:

```
function mediaAprobados(notas) {
  let suma = 0;
  let contador = 0;
  for (let i = 0; i <= notas.length; i++) {
    if (notas[i] > 5) {
      suma += notas[i];
      contador++;
    }
  }
  return suma / notas.length;
}

console.log(mediaAprobados(["7", 4, 5, 9.5, 3])); // Esperado: 7.17 aprox.
```

1. Pon un **punto de interrupción** dentro del bucle y avanza paso a paso. Vigila `i`, `notas[i]`, `suma` y `typeof suma`.
2. Localiza al menos **cuatro errores**. Anótalos en esta tabla:

Línea	Error	Cómo lo detectaste	Corrección

3. Corrige la función. Haz que devuelva `null` si no hay ninguna nota aprobada.
4. Adjunta una captura del depurador detenido en el punto de interrupción, con las variables visibles.

Parte B — Reto integrador: Biblioteca del IES

La biblioteca del instituto quiere una página para consultar su fondo: qué libros hay, cuáles están disponibles, qué géneros tiene y cuáles son los mejor valorados. Tú programas toda la lógica en JavaScript; la página solo muestra los resultados.

Código de partida

El profesor te entrega esta estructura:

```
biblioteca-ies/
├── index.html        estructura de la página, ya hecha
├── css/estilos.css   estilos mínimos, puedes ampliarlos
└── js/
    ├── datos.js      array LIBROS con 13 libros (no lo modifiques)
    └── app.js        esqueleto de las funciones que debes completar
```

Cada libro de `LIBROS` es un objeto con esta forma:

```
{ id: 4, titulo: "Fahrenheit 451", autor: "Ray Bradbury", genero: "Ciencia ficción",
  anio: 1953, prestado: true, valoraciones: [5, 5, 4, 4] }
```

`app.js` incluye una función de apoyo ya hecha, `mostrar(selector, html)`, que coloca un texto HTML dentro de un elemento de la página. Úsala como una caja negra: su funcionamiento se estudiará más adelante.

```
mostrar("#catalogo", "<p>Hola</p>");
```

Investigación previa: carga de scripts. Antes de programar, investiga cómo carga el navegador los archivos JavaScript. Responde en `respuestas.md`:

1. Consulta en MDN la página del elemento `<script>` y explica con tus palabras qué hacen los atributos `defer` y `async`. Indica en qué se diferencian.
2. Haz la prueba: quita `defer` de los dos `<script>` de `index.html`, recarga la página y copia el error que aparece en la consola. ¿Por qué se produce?
3. Vuelve a poner `defer` y cambia el orden de los scripts, de modo que `app.js` se cargue antes que `datos.js`. ¿Qué error aparece ahora y por qué?
4. ¿Por qué `defer` es más adecuado que `async` en este proyecto?

Cuando termines, deja `index.html` como estaba.

Requisitos

Las funciones R1 a R7 **calculan y devuelven** datos. Reciben el array de libros como parámetro, no usan la constante global `LIBROS` y no muestran nada por pantalla.

Req.	Función	Qué debe hacer	Comprobación con `LIBROS`
R1	`valoracionMedia(libro)`	Media de las valoraciones con un decimal, usando `reduce`. Si no tiene valoraciones, devuelve `null`.	Novelas ejemplares → `null`
R2	`librosDisponibles(libros)`	Array con los libros no prestados.	8 libros
R3	`librosEntre(libros, desde, hasta)`	Libros publicados entre dos años, ambos incluidos. Los años pueden llegar como número o como texto (`"1950"`). Si alguno no es un número válido, o `desde` es mayor que `hasta`, devuelve un array vacío.	`("1950", "1960")` → 4 libros · `("abc", 2000)` → `[]`

Req.	Función	Qué debe hacer	Comprobación con `LIBROS`
R4	`generos(libros)`	Array de géneros sin repetir, ordenado alfabéticamente. Usa un `Set`.	5 géneros, empezando por "Ciencia ficción"
R5	`librosPorAutor(libros)`	`Map` con cada autor y su número de libros.	Isaac Asimov → 2
R6	`ranking(libros, n = 3)`	Los `n` libros con mejor valoración media, de mayor a menor. Excluye los que no tienen valoraciones. Si empatan, ordena por título. **No debe modificar el array original.**	1.º Matilda (4.8)
R7	`resumen(libros)`	Objeto con `total`, `prestados`, `disponibles`, `mediaGlobal` (media de todas las valoraciones, un decimal) y `masAntiguo` (título).	`{ total: 13, prestados: 5, … masAntiguo: "La Celestina" }`

R8. Presentación. Para generar el HTML usarás **template literals** (plantillas de texto). Son cadenas escritas entre comillas invertidas (`` ` ``) que pueden ocupar varias líneas e insertar el valor de cualquier expresión con `${…}`:

```
const libro = LIBROS[0];
const fila = `
  <tr>
    <td>${libro.titulo}</td>
    <td>${libro.anio}</td>
    <td>${libro.prestado ? "Prestado" : "Disponible"}</td>
  </tr>`;
```

Es equivalente a concatenar con `+`, pero más legible: `"<td>" + libro.titulo + "</td>"`.

Escribe funciones que reciban datos y **devuelvan un string HTML** construido con template literals: la tabla del catálogo (`map` + `join`), la lista de géneros, el ranking, el resumen y los libros por autor. Después, en el programa principal, muéstralas con `mostrar` en su sección.

- En la tabla del catálogo, añade la clase `prestado` a las filas de libros prestados y dale un estilo distinto en `estilos.css`.
- Si un libro no tiene valoraciones, muestra `Sin valorar` en lugar de `null`.

R9. Calidad del código.

- Solo `const` y `let`; nunca `var`.
- Nombres descriptivos y funciones cortas, con una sola responsabilidad.
- Un comentario de cabecera en cada función: qué hace, qué recibe y qué devuelve. Evita comentarios que repitan lo que ya dice el código.

R10. Pruebas y depuración.

- En `js/pruebas.js`, comprueba cada función R1 a R7 con `console.assert` usando los valores de la tabla. Incluye al menos un caso límite por función (array vacío, año no válido, `n` mayor que el número de libros…).
- Muestra el resultado de `librosDisponibles` con `console.table`.
- Adjunta una captura del depurador detenido dentro de `ranking`, con la función de comparación de `sort` en ejecución.

Restricciones

- R1 a R7 se resuelven con métodos de arrays (`filter`, `map`, `reduce`, `sort`, `forEach`), `Set` y `Map`. No uses bucles `for` en estas funciones.
- No uses librerías ni frameworks.
- No modifiques `datos.js` ni la función `mostrar`. No uses otras instrucciones para modificar la página.

Entrega y evidencias

Se entrega la URL del repositorio y un único `.zip` en el aula virtual con el nombre `apellido_nombre_biblioteca.zip` y esta estructura:

```
apellido_nombre_biblioteca/
├── parteA/
│   ├── index.html
│   └── ejA1.js … ejA6.js
├── biblioteca-ies/
│   ├── index.html       (con js/pruebas.js añadido después de app.js)
│   ├── css/estilos.css
│   └── js/datos.js, app.js, pruebas.js
├── capturas/
│   ├── A6-depurador.png
│   └── B-ranking-depurador.png
└── respuestas.md        tablas y respuestas de la parte A y pregunta sobre defer
```

Antes de entregar, comprueba que:

- ☐ La página se abre sin errores en la consola.
- ☐ Todas las comprobaciones de `pruebas.js` pasan (ningún `Assertion failed`).
- ☐ Todas las funciones tienen su comentario de cabecera.
- ☐ Las dos capturas muestran el depurador detenido y las variables visibles.

El profesor podrá pedirte que expliques o modifiques en clase alguna parte de tu código.

Refuerzo y ampliación

Refuerzo

Puedes usar estas pistas, de menos a más concretas:

Req.	Pista 1	Pista 2
R1	Primero suma, luego divide entre la cantidad de valoraciones.	`reduce((acumulado, valor) => acumulado + valor, 0)`. Para un decimal: `Math.round(x * 10) / 10`.
R3	Convierte los parámetros antes de comparar.	`Number("1950")` devuelve `1950`; `Number("abc")` devuelve `NaN`.
R4	Un `Set` no admite duplicados.	`[...new Set(array)]` convierte el conjunto de nuevo en array.
R5	Recorre los libros y suma 1 al contador del autor.	`mapa.get(autor) ?? 0` devuelve 0 la primera vez.
R6	`sort` modifica el array sobre el que se llama.	Ordena una copia: `[...libros]`. Para los empates, mira `localeCompare`.

Ampliación

Opcionales; suman en la valoración del nivel Excelente.

1. **Ranking configurable.** Añade a `ranking` un tercer parámetro: una función que indique el criterio de orden (por media, por número de valoraciones, por antigüedad…).
2. **Búsqueda por texto.** Escribe `buscar(libros, texto)`, que encuentre coincidencias en título o autor sin distinguir mayúsculas.
3. **Métodos recientes del lenguaje.** Investiga en MDN `toSorted()` (ES2023) y `Map.groupBy()` (ES2024). Úsalos para simplificar R6 y para agrupar los libros por género. Explica en `respuestas.md` qué aportan frente a tu solución.

> ***Contenido de actualización/ampliación respecto a la programación.*** *El punto 3 no figura en la programación. Se incluye porque son métodos estándar actuales que resuelven de forma más segura problemas trabajados en la UT4 (ordenar sin modificar el original y agrupar).*